

## Django project 6 : Model and Dynamic URL(Findmusic app)

### Step 1: create project

```
python -m django startproject project6
```

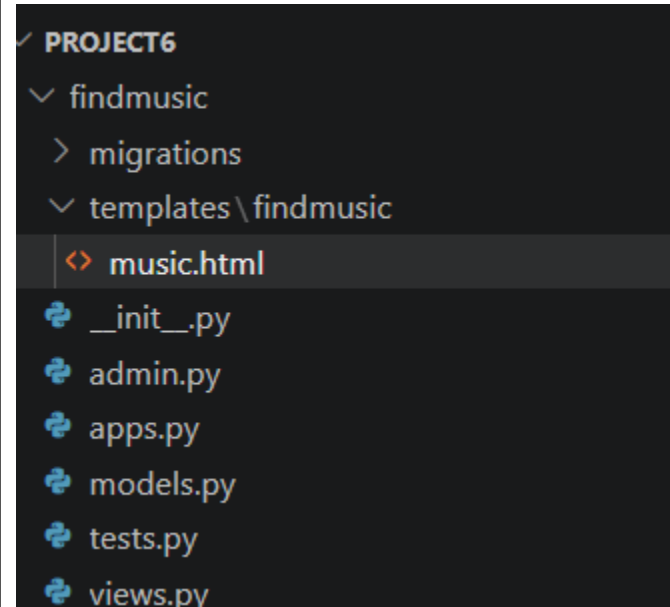
### Step 2: create application find music

```
cd project6  
python manage.py startapp findmusic
```

### Step 3: Install the app in setting.py

```
'findmusic',
```

### Step 4 : Create Templates Folder and then Create findmusic folder in Templates and then make music.html file



```
✓ PROJECT6  
  ✓ findmusic  
    > migrations  
  ✓ templates\findmusic  
    <> music.html  
  + __init__.py  
  + admin.py  
  + apps.py  
  + models.py  
  + tests.py  
  + views.py
```

### Step 5 : Create a model for Music

findmusic->[models.py](#)

```
models.py X
findmusic > models.py > Music
1  from django.db import models
2
3  # Create your models here.
4  class Music(models.Model):
5      song = models.CharField(max_length=225)
6      artist = models.CharField(max_length=225)
7      year = models.IntegerField()
8      album = models.CharField(max_length=225)
```

### Step 6 : Register Model in admin.py

findmusic->admin.py

```
admin.py X
findmusic > admin.py
1  from django.contrib import admin
2  from .models import Music
3
4  # Register your models here.
5  admin.site.register(Music)
```

### Step 7 : In cmd -> python [manage.py](#) makemigrations

```
C:\Users\patel\DJANGO\project6>python manage.py makemigrations
Migrations for 'findmusic':
  findmusic\migrations\0001_initial.py
  + Create model Music
```

Step 8 : In cmd - > python [manage.py](#) migrate

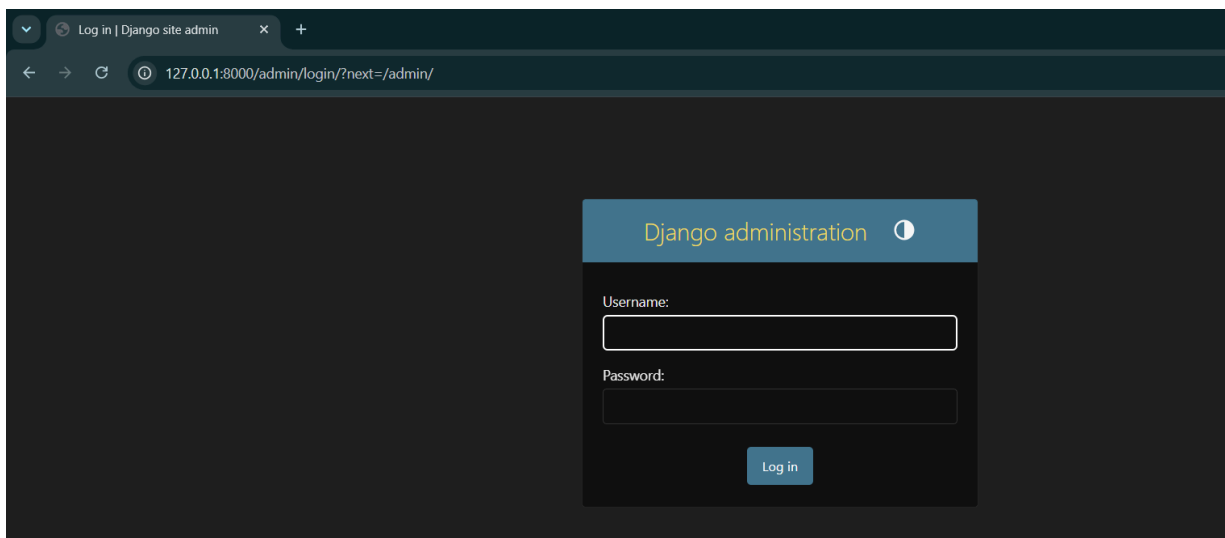
```
C:\Users\patel\DJANGO\project6>python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, findmusic, sessions
Running migrations:
  Applying contenttypes.0001 initial... OK
```

Step 8 : In cmd - > python [manage.py](#) createsuperuser

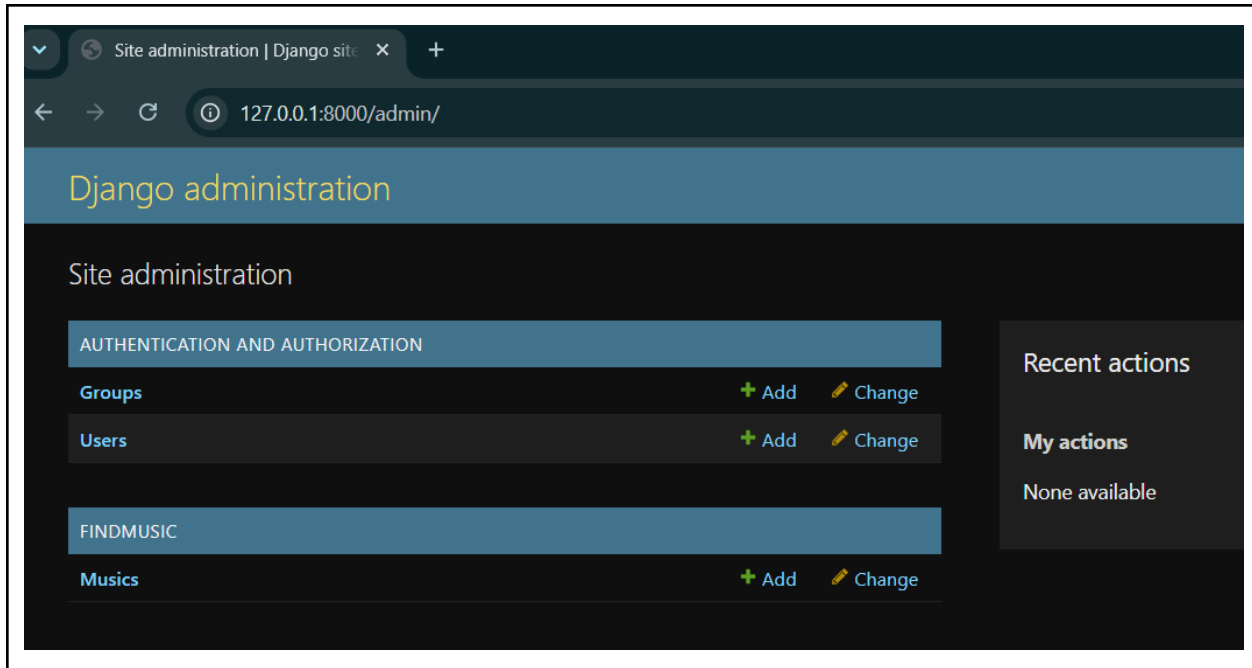
```
C:\Users\patel\DJANGO\project6>python manage.py createsuperuser
Username (leave blank to use 'patel'): pritu
Email address: pritu@gmail.com
Password:
Password (again):
This password is too short. It must contain at least 8 characters.
This password is entirely numeric.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.
```

Step 9 : In cmd-> python [manage.py](#) runserver

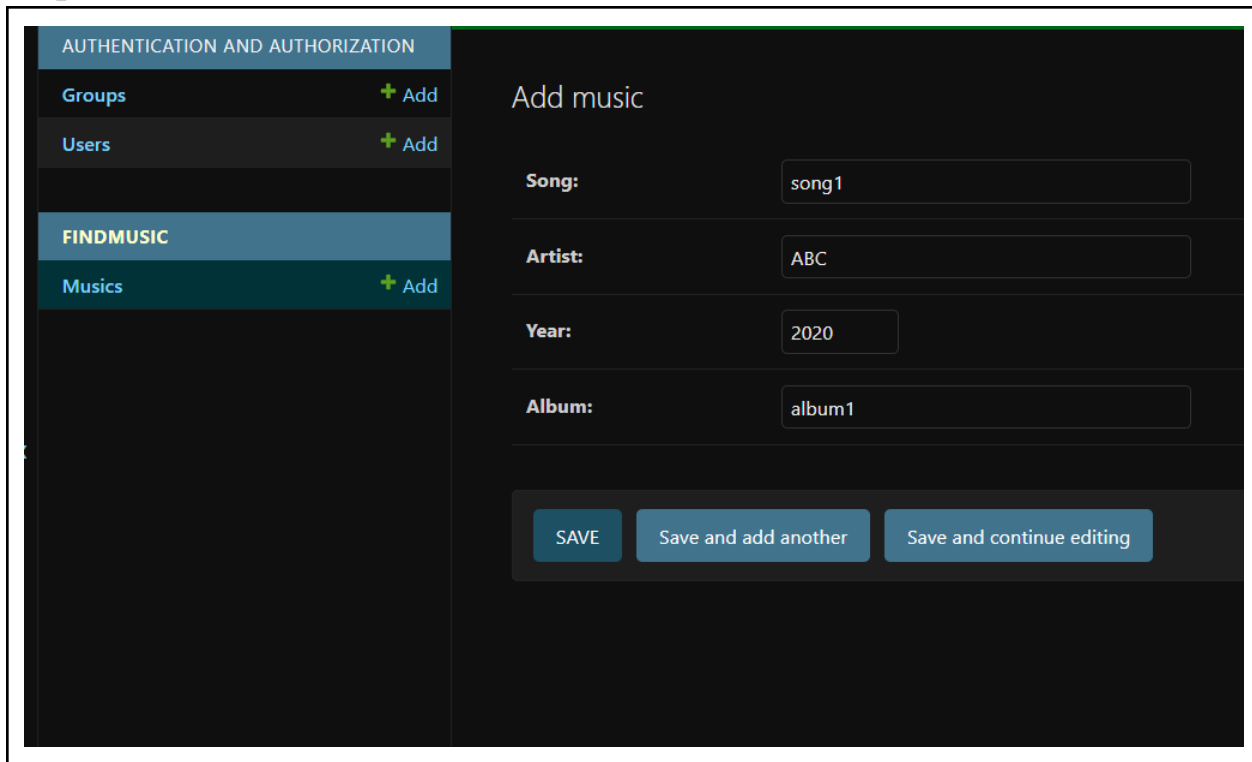
Open link : (<http://127.0.0.1:8000/>)

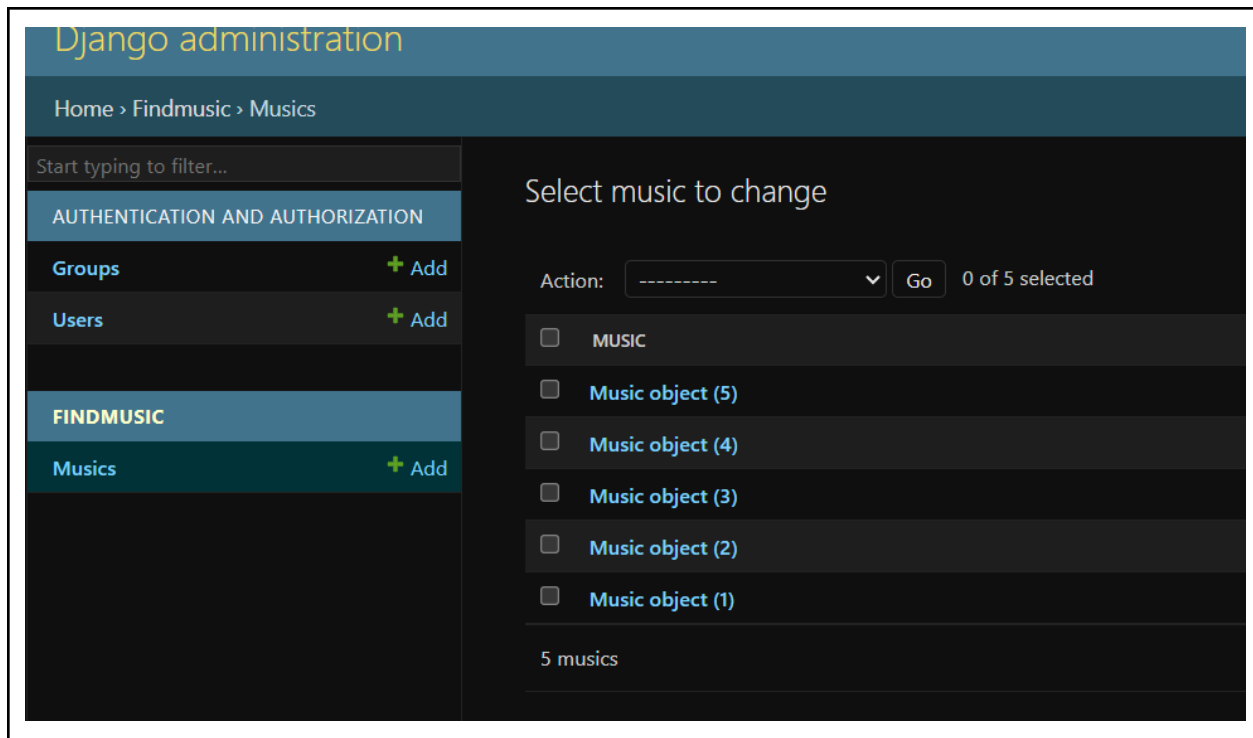


## Step 10 : write username and password and login them



## Step 11 : Add music





## Task : Search Music and Show list of Music

### Step 12 : make form in music.html file

findmusic app->templates folder ->findmusic folder ->music.html file

```
music.html X
findmusic > templates > findmusic > music.html > html > body
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Find Music</title>
7 </head>
8 <body>
9   <form action="" method="POST">
10
11     <h1>Welcome</h1>
12     <h2>Let's search your music!</h2>
13
14     <label for="song">Song</label>
15     <input type="text" name="song" id="song" class="song">
16
```

```

16
17     <label for="artist">Artist</label>
18     <input type="text" name="artist" id="artist" class="artist">
19
20     <label for="year">Year</label>
21     <input type="number" name="year" id="year" class="year">
22
23     <label for="album">Album</label>
24     <input type="text" name="album" id="album" class="album">
25
26     <button type="submit">Submit</button>
27 </form>
28 </body>
29 </html>

```

**Step 13 : render music.html file in views.py**  
**findmusic app->**[views.py](#)

```

views.py  X
findmusic > views.py > music
1  from django.shortcuts import render
2
3  # Create your views here.
4  def music(request):
5      return render(request, "findmusic/music.html")

```

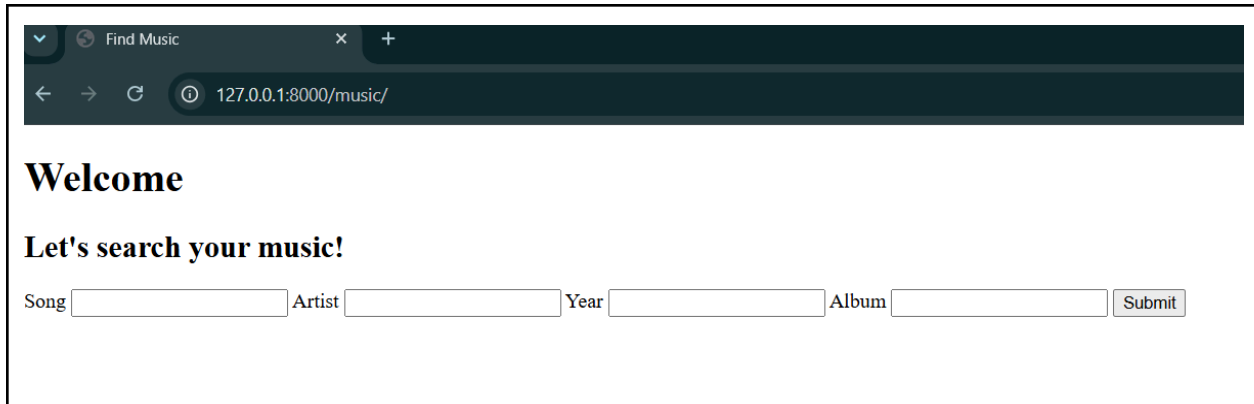
**Step 14: connect to the url in [urls.py](#)**  
**project6 ->** [urls.py](#)

```

urls.py  X
project6 > urls.py > ...
17  from django.contrib import admin
18  from django.urls import path
19  from findmusic.views import music
20
21  urlpatterns = [
22      path('admin/', admin.site.urls),
23      path('music/', music),
24  ]

```

## Output :



Find Music

127.0.0.1:8000/music/

## Welcome

Let's search your music!

Song  Artist  Year  Album

## Step 15 : Show the list of music

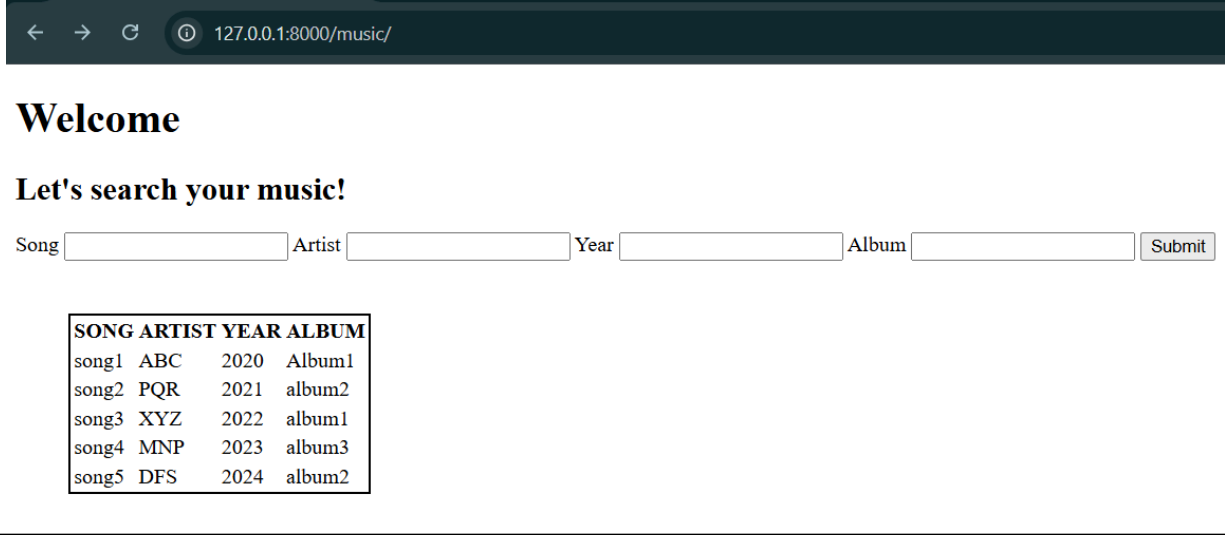
**music.html**

```
views.py  music.html X
findmusic > templates > findmusic > music.html > html > body > div
 2  <html lang="en">
 8  <body>
 9    <form action="" method="POST">
25      <button type="submit">Submit</button>
26    </form>
27
28    <div style="margin: 40px ;">
29      <table style="border: 2px solid black;">
30        <tr>
31          <th>SONG</th>
32          <th>ARTIST</th>
33          <th>YEAR</th>
34          <th>ALBUM</th>
35        </tr>
36        {% for music in music_records %}
37          <tr>
38            <td> {{music.song}}</td>
39            <td> {{music.artist}}</td>
40            <td> {{music.year}}</td>
41            <td> {{music.album}}</td>
42          </tr>
43        {% endfor %}
44      </table>
45    </div>
46  </body>
47 </html>
```

**views.py**

```
views.py x music.html
findmusic > views.py > music
1 from django.shortcuts import render
2 from .models import Music
3
4 # Create your views here.
5
6 def music(request):
7     music_records = Music.objects.all()
8     return render(request, "findmusic/music.html", {'music_records': music_records})
```

output:



Step 16: Submitting Data and Search Music

{% csrf\_token %}

- A CSRF token (Cross-Site Request Forgery token) is a unique, secret, and unpredictable string generated by a server and given to a user's browser.
- It acts as a security "password" ensuring that form submissions and sensitive actions come from the legitimate user, not a malicious third-party site.

music.html



```
views.py  music.html X
findmusic > templates > findmusic > <> music.html
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Find Music</title>
7  </head>
8  <body>
9      <form action="" method="POST">
10         {% csrf_token %}
```

## views.py

```
views.py X  music.html
findmusic > views.py > music
1  from django.shortcuts import render
2  from .models import Music
3
4  # Create your views here.
5
6  def music(request):
7      music_records = Music.objects.all()
8
9      # Apply filters if the form submitted
10     if request.method == 'POST':
11         song = request.POST.get('song')
12         artist = request.POST.get('artist')
13         year = request.POST.get('year')
14         album = request.POST.get('album')
15
16         if song:
17             music_records = music_records.filter(song__icontains=song)
18
19         if artist:
20             music_records = music_records.filter(artist__icontains=artist)
21
22         if year:
23             music_records = music_records.filter(year__icontains=year)
24
25         if album:
26             music_records = music_records.filter(album__icontains=album)
27
28     return render(request, "findmusic/music.html", {'music_records': music_records})
```

Output:

Find Music

127.0.0.1:8000/music/

## Welcome

Let's search your music!

Song  Artist  Year  Album

| SONG  | ARTIST | YEAR | ALBUM  |
|-------|--------|------|--------|
| song1 | ABC    | 2020 | Album1 |

```
music_records = music_records.filter(song__icontains=song)
```

**music records:** Your original dataset or list of records.

**.filter(...):** A method that iterates through the data, keeping only rows that meet a specific condition.

**song\_\_icontains = song:** The filtering condition.

- **song:** The specific column/field in your data holding track names.
- **\_\_icontains:** The lookup operator. The *i* stands for *case-insensitive*, and contains checks if the substring exists within the string.
- **=song:** The variable, string, or user input you are searching for